Bit Manipulation

uint8_t reg = 0b01011010

Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0	1	1	0	1	0	1	0

Bit 0 → LSB (Least Significant Bit)
Bit 7 → MSB (Most Siginificant Bit)

*Mask:

binary value used to control or access specific bit in a register.

reg = 0b01101010

Bit	Mask	
0	0b0000 0001	1<<0
1	0b0000 0010	1<<1
2	0b0000 0100	1<<2
3	0b0000 1000	1<<3
4	0b0001 0000	1<<4
5	0b0010 0000	1<<5
6	0b0100 0000	1<<6
7	0b1000 0000	1<<7
3,4,5	0b0011 1000	1<<3 | 1<<4 | 1<<5
6,7	0b1100 0000	1<<6 | 1<<7

or

① Set Bit:

make a specific bit = 1

⇒ Steps:

① shift 1 to the left of the bit number we want to make High (1), thus creating the mask.

1 << Bit_Num

①

② Mask OR Reg.

Reg ← Reg OR Mask

⇒ Example:

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	
reg = 0b	1	1	1	0	0	0	0	0	, set Bit 3.

Mask = 1 << 3 ⟹ {1} shift left By 3

Mask = 0b 0000 1000

Now: reg = reg OR Mask

remark

X or 0 = X

X or 1 = 1

```
  0b 1110 0000
  0b 0000 1000   OR
  ------------
  0b 1110 1000
```

→ Now Bit 3 = 1 ✓

* Code C:

```
uint8_t reg = 0b1110 0000;
uint8_t mask = 1 << 3;
        reg = reg | mask;
```

```
reg |= (1<<3);
```

② Clear Bit:

make specific Bit = 0;

⇒ Steps:

① 1 shift left By Bit Num. (Mask) (1 << Bit)

② Complement Mask. ~(1 << Bit)

③ Mask AND Reg. Reg = Reg & ~(1 << Bit);

* Example:

reg = 0b 1101 1110 , clear Bit 2?

Mask = 1 << 2 ; Mask = 0b 0000 0100
Mask = ~Mask ; Mask = 0b 1111 1011

remark
x AND 0 = 0
x AND 1 = x

reg = reg AND Mask ; 0b 1101 1110
0b 1111 1011 AND
1101 1010
clear Bit 2

* C code :

```
uint8_t reg = 0b 1101 1110
uint8_t mask = 1 << 2;
        mask = ~ mask;
        reg = reg & mask
```

$\boxed{\text{reg \&= ~(1 << 2)}}$

the same

---

③ Toggle Bit:

logical

if Bit = 1 ⟶ Bit = 0 | XOR 0 = 1
if Bit = 0 ⟶ Bit = 1 | XOR 1 = 0

* Steps:
① 1 shift By Bit_num (Mask) 1 << Bit
② Mask XOR Reg.

⇒ Example:
reg = 0b 1101 0010 , toggle Bit 5.

Mask = 1 << 5 ; Mask = 0b 0010 0000
reg = reg XOR Mask ;

0b 1101 0010
0b 0010 0000 XOR
0b 1111 0010
Bit 5 = 1

* C code:

```
uint8_t reg = 0b1101 0010;
uint8_t mask = 1<<5;
        reg = reg ^ mask;
```

the same

$\boxed{\text{reg ^= 1<<5;}}$

④ Get Bit:

Read value of bit (0 or 1)

* Steps:
  1. 1 shift left By Bit Num. (Mask) 1<< Bit
  2. Mask AND Reg.
  3. (Mask AND Reg) ⇒ shift Right By Bit.

⇒ Example:

reg = 0b 1110 0111 , get value of Bit 6;

```
Mask = 1 << 6;
bit  = reg. AND Mask
bit  = Bit >> 6;
```

```
  0b 1110 0111
  0b0100 0000   AND
  ------------
  0b0100 0000   >> 6
  0b0000 0001 → the value = 1
```

* C code:

```
unit8_t reg = 0b 1110 0111;
unit8_t mask = 1 << 6;
unit8_t value = reg & mask;
        value = value >> 6;
```

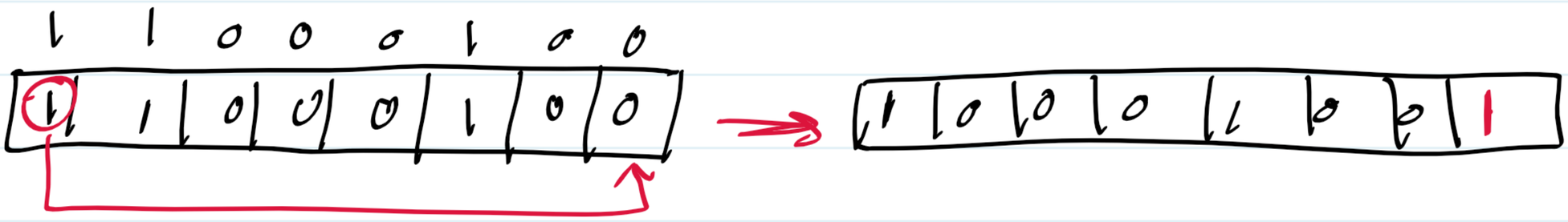
```
value = (reg&(1<<6)) >> 6;
```

the same

---

⑤ ROL (Rotate Left):

shif left + wrap around.



* Steps:
  1. shift reg to the left by the number of turns. reg << N
  2. shift reg to the right by the (reg size - N). reg >> (size - N)
  3. step① OR step②

→ Example:

reg = ob 1101 0000 , ROL By 1

vegestar_size = 8_bit.

r1 = reg << 1 → ob 1010 0000

r2 = reg >> (8 - 1) →

reg >> 7

$\boxed{\text{reg} = r1 \mid r2}$

befor →

1	1	0	1	0	0	0	0
1	1	1	0	1	0	0	0
2		1	1	0	1	0	0
3			1	1	0	1	0
4				1	1	0	1
5					1	1	0
6						1	1
7	0	0	0	0	0	0	0	1

* C code;

```
uint8_t reg = 0b 1101 0000;
uint8_t r1 = reg << 1;
uint8_t r2 = reg >> 7;
reg = r1 | r2;
```

```
reg = (reg<<1) | (reg>>7);
```

the same

6 ROR (Rotate Right):

Shift right + wrap_around

*Steps:

1. shift reg to right By the number of turn.
2. shift reg to left By (reg_size - # of turn (N)).
3. step1 or step2

* Example:

reg = ob 1011 0000 , ROR By 2 → N

r1 = reg >> 2 ; ob 0010 1100

r2 = reg << (8-2)

reg << 6

reg = r1 | r2;

* C code:

```
uint8_t reg = 0b 1011 0000;
uint8_t r1 = reg >> 2;
uint8_t r2 = reg << 6;
reg = r1 | r2;
```

1	0	1	1	0	0	0	0	befor
0	1	1	0	0	0	0		1
1	1	0	0	0	0			2
1	0	0	0	0				3
0	0	0	0					4
0	0	0						5
0	0	0	0	0	0	0	0	6

5